CS6530 — APPLIED CRYPTOGRAPHY July-Nov 2026

Assignment 2
Authenticated Ephemeral Key Establishment

X25519 • Ed25519 • SHA-256 • HKDF-SHA-256 • AES-256-GCM

Central task: Alice and Bob run on two different computers connected through an untrusted network. Establish fresh authenticated session keys, exchange protected application messages, and experimentally demonstrate MITM resistance, replay resistance, and forward secrecy.

Assignment at a Glance

1. Normal Session	2. MITM	3. Replay	4. Forward Secrecy
Authenticated handshake + protected data	Show attack without identity authentication	Replay a captured application record	Record now; compromise long-term key later

Scope boundary: This is a cryptography assignment, not a networking assignment. A minimal TCP transport helper is supplied. Use standard cryptographic libraries; do not implement cryptographic primitives yourself.

1. Learning Outcomes

- Distinguish long-term identity authentication from ephemeral key agreement.
- Build an authenticated ephemeral X25519 handshake using Ed25519 signatures.
- Derive independent directional traffic keys using HKDF-SHA-256.
- Protect application records using AES-256-GCM with AAD and monotonic counters.
- Experimentally explain why authentication prevents MITM, why replay state is needed, and why ephemeral secrets provide forward secrecy.

2. Fixed Cryptographic Suite

Primitive	Purpose
X25519	Fresh ephemeral shared-secret establishment
Ed25519	Long-term identity authentication
SHA-256	Handshake transcript hash
HKDF-SHA-256	Session-key derivation and domain separation
AES-256-GCM	Confidentiality and integrity of application data

Important: X25519 and Ed25519 use separate key pairs for separate purposes. Do not reuse or convert one key pair for the other. Assume Alice and Bob already possess authentic copies of each other's long-term Ed25519 public identity key.

3. System and Transport Model

Team setup. Each team of two runs Alice on one physical computer and Bob on another. The machines communicate over an ordinary/untrusted IP network. Mallory may be implemented as an additional process/component; a third physical machine is not required.

Supplied transport helper. The helper provides a persistent TCP connection and message framing only. It provides no confidentiality, authentication, integrity protection, or replay protection.

Instructor-supplied helper files. The assignment is distributed with transport.py, alice_test.py and bob_test.py. transport.py provides the persistent TCP connection and 2-byte length || message framing. alice_test.py and bob_test.py are simple connectivity tests to verify communication between the two machines before implementing the cryptographic protocol.

Use of the helper. First verify that the supplied test programs communicate successfully between the two machines. Then use transport.py to exchange M1-M4 and protected application records. Students are not required to modify transport.py; all cryptographic functions remain part of the team implementation.

On the wire: uint16_be(message_length) || message_bytes

Student interface: connect_to_peer()/accept_peer(), send_message(sock, bytes), receive_message(sock)

Before cryptography: First run the supplied Alice/Bob connectivity test and verify that a plain 'Hello Bob' / 'Hello Alice' exchange works between the two machines.

4. Protocol Parameters and Canonical Encoding

Field	Size	Encoding / value
Protocol_ID	12 bytes	ASCII "CS6530-A2-v1"
Alice_ID	8 bytes	Official 8-character roll number, ASCII
Bob_ID	8 bytes	Official 8-character roll number, ASCII
Alice_SID	16 bytes	Fresh random session identifier
Bob_SID	16 bytes	Fresh random session identifier
Alice_Ephemeral_PK	32 bytes	Raw X25519 public key
Bob_Ephemeral_PK	32 bytes	Raw X25519 public key

Canonical handshake transcript (all fields have fixed sizes):

Protocol_ID || Alice_ID || Bob_ID || Alice_SID || Bob_SID || Alice_Ephemeral_PK || Bob_Ephemeral_PK

Total canonical transcript length = 124 bytes. Transcript_Hash = SHA-256(transcript) = 32 bytes (256 bits).

Transcript_Hash = SHA-256(Handshake_Transcript)

Serialization: Network messages may use a simple student-defined serialization (for example, JSON with binary fields represented in hex). For cryptographic hashing/signing and AAD, decode fields to raw bytes and construct exactly the canonical byte strings specified here.

5. Protocol Walkthrough — Authenticated Ephemeral Handshake

ALICE		BOB
Generate Alice_SID Generate ephemeral X25519 key pair	M1: Alice_SID, Alice_Ephemeral_PK ——————▶	
	◀—————— M2: Alice_SID, Bob_SID, Bob_Ephemeral_PK	Generate Bob_SID Generate ephemeral X25519 key pair
Check returned Alice_SID		
Construct transcript SHA-256 → Transcript_Hash		Construct same transcript SHA-256 → Transcript_Hash
Sign Transcript_Hash	M3: Alice_Signature ————————————▶	Verify using Alice LT public key
Verify using Bob LT public key	◀———————————— M4: Bob_Signature	Sign Transcript_Hash
AUTHENTICATED HANDSHAKE COMPLETE		AUTHENTICATED HANDSHAKE COMPLETE

Figure 1. Baseline four-message authenticated handshake.

Signature operations:

Alice_Signature = Ed25519_Sign(Alice_LT_PrivateKey, Transcript_Hash)
Bob_Signature = Ed25519_Sign(Bob_LT_PrivateKey, Transcript_Hash)

Failure rule: If the echoed Alice_SID is wrong or either Ed25519 signature verification fails, abort the session. No application data is accepted.

6. Functional Requirements (FR)

Implement the following functional requirements. The ladder in Section 5 explains the handshake flow; the requirements below form the implementation checklist.

FR-1 — Two-machine session and identities. Run Alice and Bob on two physical computers using the supplied TCP helper. Configure each side with its own long-term Ed25519 identity key pair and an authentic copy of the peer's long-term Ed25519 public key.

FR-2 — Fresh session values. For every new session, Alice generates a fresh 16-byte Alice_SID and ephemeral X25519 key pair; Bob generates a fresh 16-byte Bob_SID and ephemeral X25519 key pair. M2 must echo Alice_SID and Alice must verify it.

FR-3 — Canonical transcript and mutual authentication. Construct exactly the canonical 124-byte transcript defined in Section 4, compute its SHA-256 hash, sign the hash with the sender's long-term Ed25519 private key, and verify the peer signature using the trusted long-term public key. Abort on SID mismatch or signature failure.

FR-4 — Shared secret and traffic-key derivation. Compute the X25519 shared secret and derive K_Alice_to_Bob and K_Bob_to_Alice using HKDF-SHA-256 with Transcript_Hash as salt and the specified info strings. Do not use the raw X25519 shared secret directly as an AES key.

FR-5 — Protected bidirectional communication. Use AES-256-GCM for application data. Maintain a separate uint64 counter per direction, starting at 0; derive the 96-bit nonce as 0x00000000 || uint64_be(counter); construct AAD exactly as specified; use the correct directional key.

FR-6 — Replay state and failure handling. For this assignment, accept only the expected-next receive counter. Reject stale/unexpected counters, failed AEAD verification, invalid handshake authentication, or malformed required protocol fields. Do not continue a failed authenticated session.

FR-7 — Ephemeral-secret lifecycle. After a completed session, discard ephemeral X25519 private keys, the X25519 shared secret, and derived traffic keys when they are no longer needed. A controlled copy may be retained only for the forward-secrecy comparison in TR-4.

Reference formulas:
PRK = HKDF-Extract(salt = Transcript_Hash, IKM = Shared_Secret)
K_Alice_to_Bob = HKDF-Expand(PRK, info = "CS6530-A2 Alice->Bob", L = 32)
K_Bob_to_Alice = HKDF-Expand(PRK, info = "CS6530-A2 Bob->Alice", L = 32)
AAD = Sender_ID || Receiver_ID || Alice_SID || Bob_SID || Message_Counter

7. Test Requirements (TR)

Demonstrate the following tests and retain clear evidence. The tests verify that the functional requirements behave as intended under normal and adversarial conditions.

TR-1 — Normal authenticated session. Complete M1–M4, derive the two directional traffic keys, and exchange at least three protected application messages in each direction. Show successful signature/AEAD verification, counters 0,1,2, and that the raw X25519 shared secret is not directly used as the AES-GCM key.

ALICE		**BOB**
Complete M1–M4	Authenticated handshake	Complete M1–M4
Derive directional keys		Derive same keys
Counter 0,1,2	AEAD records ——————▶	Verify + decrypt
Verify + decrypt	◀—————— AEAD records	Counter 0,1,2

Figure 2. TR-1: Normal authenticated and protected session.

TR-2 — MITM experiment. First run an intentionally weakened mode with Ed25519 authentication disabled. Mallory substitutes ephemeral public keys, establishes separate Alice–Mallory and Mallory–Bob secrets, decrypts/modifies/re-encrypts a message. Then restore full authentication and show that the same substitution causes signature verification failure and session abort.

ALICE	MALLORY	**BOB**
Alice ephemeral PK	—▶ substitute ephemeral PK —▶	
	◀— substitute ephemeral PK —	Bob ephemeral PK
Secret A–M	decrypt / modify / re-encrypt	Secret M–B
Full protocol restored	signature verification fails	SESSION ABORTED

Figure 3. TR-2: MITM succeeds without identity authentication and fails when authentication is restored.

TR-3 — Replay experiment. Capture a valid AES-GCM application record and replay it unchanged. Show that the original record is accepted and the replay is rejected because its counter is stale. Explain why an unchanged replay may still have a valid GCM tag and why freshness therefore requires replay state.

ALICE	MALLORY / NETWORK	BOB
Send counter = 0	—▶ capture ————▶	Expected 0 → ACCEPT
	Replay same record ———▶	Expected 1; got 0 → REJECT

Figure 4. TR-3: Replay of a previously valid application record.

TR-4 — Forward-secrecy experiment. Run and record a normal session S1, then discard ephemeral/session secrets. Later simulate compromise of Alice's long-term Ed25519 private key and explain/show why recorded S1 traffic keys cannot be reconstructed. For controlled comparison, demonstrate that possession of the old ephemeral X25519 private key would permit reconstruction.

SESSION S1	ATTACKER	LATER
Run normal session	Record handshake + ciphertext	
Discard ephemeral/session secrets		
	Obtain Alice long-term Ed25519 private key	
Past S1 remains confidential	Old X25519 shared secret not reconstructed	

Figure 5. TR-4: Save now, compromise the long-term authentication key later.

Evidence/logging expectation. Design clear and sufficiently detailed logs; no single mandatory log format is prescribed. A strong log should make the protocol execution and test outcome easy to verify: peer identity, SIDs, message direction/type, counter/nonce, AAD, ciphertext/tag or concise fingerprints where appropriate, and explicit SUCCESS / FAILED / REJECTED results. More complete and useful evidence will receive stronger credit within the evidence/report component.

8. Evidence and Deliverables

- Source code, including clear Alice, Bob and Mallory/test components.
- README with exact commands and steps to run the protocol on two physical machines.
- Short report explaining M1–M4, transcript construction, X25519/HKDF key derivation, AES-GCM AAD/nonce/counter design, and the four demonstrations.
- Selected logs/screenshots that clearly show the normal session, MITM result, replay rejection and forward-secrecy investigation.
- Individual viva: each student must be able to explain the protocol, code and security reasoning.
- AI-use disclosure consistent with the course assignment policy.

9. Marking Weightage — 100 Marks

Component	What is assessed	Marks
A. FR: Authenticated handshake	Correct M1–M4; fresh SIDs/ephemeral keys; transcript; Ed25519 sign/verify; abort on failure	20
B. FR: Key derivation	Correct X25519 shared secret; HKDF-SHA-256; transcript binding; separate directional 256-bit keys	15

C. FR: Protected communication	AES-256-GCM; correct nonce/counter/AAD; bidirectional records; successful verification/decryption	15
D. TR-2: MITM demonstration	Successful attack in weakened mode; failure in authenticated mode; correct explanation	15
E. TR-3: Replay demonstration	Capture/replay; counter-based rejection; correct freshness reasoning	10
F. TR-4: Forward secrecy	Save-now/compromise-later experiment; deletion of ephemeral secrets; correct reasoning/comparison	10
G. Code, evidence and report	Readable code; reproducible two-machine instructions; clear and sufficiently detailed logs/screenshots that support the claimed results; concise technical explanation	10
H. Individual viva	Individual understanding of design choices, implementation and observed security behaviour	5

Total: 100 marks

10. Curiosity Bonus — Observe the Protocol on the Wire

Optional: up to 20% bonus relative to Assignment 2 marks. Capture one successful Alice–Bob session using Wireshark and relate the capture to both Section 5 (the M1–M4 authenticated handshake) and Section 7 (protected application communication). Identify M1–M4 and at least one AES-256-GCM protected application record. Show which fields are visible on the network (for example IDs, SIDs, counters/nonces, ciphertext/tag) and which information is not visible. Briefly explain why AAD may be visible while still being integrity protected.

The bonus rewards curiosity and correct interpretation; elaborate packet-analysis tooling is not required.

11. Security and Implementation Boundaries

- Use standard cryptographic libraries. Do not implement X25519, Ed25519, SHA-256, HKDF or AES-GCM primitives yourself.
- Do not use TLS or another secure channel to solve the assignment; the objective is to build and observe this protocol.
- Do not add certificate/PKI complexity. Peer long-term Ed25519 public keys are assumed to be authentic for this assignment.
- No GUI, database, asynchronous framework or multi-client server is required.
- Do not log or submit real private credentials unrelated to this assignment.
- Marks are for cryptographic correctness, experimental evidence and security reasoning—not for UI/networking sophistication.

12. One-Sentence Protocol Story

Exchange fresh temporary public keys → authenticate those keys → independently create the same shared secret → derive directional session keys → securely communicate.